UNIT 20 90 min

# PyTorch: Tensors, Gradients, and Neural Networks

Tensors, the operations on them, automatic differentiation, and building a small neural network with nn.Module.

---

### By the end of this unit you will be able to

- Create tensors and check their shape, dtype, and device
- Explain how a tensor relates to a NumPy array, and convert between them
- Use `requires_grad` and `backward()` to compute gradients automatically
- Explain what a neuron, a layer, and an activation function do
- Define a small network with `nn.Module` and run a forward pass

**Note**

**Where to run this unit.** PyTorch is not available in the browser sandbox. Use Google Colab (free, no installation; set Runtime → Run all) or install PyTorch locally with `pip install torch`. Every code block here was run with PyTorch 2.x on CPU. Output shown in the text may differ slightly in the last decimal place.

## Tensors

A **tensor** is an n-dimensional array, like a NumPy array, with two extra abilities: it can live on a GPU, and it can track how it was computed so gradients can be found automatically. Most of the code you write looks like NumPy.

tensors.py

```
import torch

x = torch.tensor([[1.0, 2.0], [3.0, 4.0]])
print(x)
print(x.shape, x.dtype)
print(torch.zeros(2, 3))
print(torch.arange(5))
```

| NumPy | PyTorch | Notes |
|---|---|---|
| `np.array(...)` | `torch.tensor(...)` | Copies the data |
| `np.zeros(...)` | `torch.zeros(...)` | Same idea |
| `a.shape` | `t.shape` | Same idea |
| `a.dtype` | `t.dtype` | Default float is float32 in PyTorch, float64 in NumPy |
| `a @ b` | `t @ u` | Matrix product works the same way |

**Watch out**

Tensors and arrays must have compatible dtypes for many operations. A common error is mixing `int64` with `float32`. Convert with `t.float()` when a function asks for floats.

convert.py

```
import numpy as np
import torch

a = np.array([1.5, 2.5])
t = torch.from_numpy(a)        # shares memory with a
back = t.numpy()               # back to NumPy
print(t, back)
```

## Operations and devices

Operations work element by element, use broadcasting, and matrix products with `@`, exactly as in NumPy. Tensors can move to a GPU with `.to(device)`. Code that picks the device at the top keeps working on machines without a GPU.

device.py

```
import torch

device = "cuda" if torch.cuda.is_available() else "cpu"
x = torch.ones(2, 3, device=device)
w = torch.full((3, 2), 0.5, device=device)
print(device, (x @ w).shape)
```

## Automatic gradients

Training a model means adjusting numbers to reduce an error. The adjustment is the **gradient**: how much the error changes when each number changes. PyTorch computes gradients automatically. Set `requires_grad=True` on the tensors you want gradients for, compute a result, then call `backward()` on it. The gradients appear in `.grad`.

autograd.py

```
import torch

x = torch.tensor(3.0, requires_grad=True)
y = x ** 2 + 2 * x      # y = x^2 + 2x

y.backward()            # compute dy/dx
print(x.grad)           # 2x + 2 at x = 3 is 8
```

**Trace: a gradient by hand** TRACE

```
x = 3.0
y = x ** 2 + 2 * x
slope = 2 * x + 2
print(slope)
```

| Step | Line run | Variables after the line | Output so far |
|---|---|---|---|
| 1 | 1 | `x = 3.0` | — |
| 2 | 2 | `x = 3.0, y = 15.0` | — |
| 3 | 3 | `x = 3.0, y = 15.0, slope = 8.0` | — |
| 4 | 4 | `x = 3.0, y = 15.0, slope = 8.0` | `8.0` |

Gradients accumulate in `.grad`. Clear them before each new step with `optimizer.zero_grad()` (Unit 21), or the numbers add up across steps.

## A neuron and an activation

A **neuron** multiplies each input by a weight, adds a bias, and passes the total through an **activation function**. Without an activation, stacked layers collapse into one linear map. `ReLU` is the usual choice: it outputs the input if it is positive, and 0 otherwise. Non-linear activations let networks learn curves.

relu.py

```
import torch
import torch.nn as nn

relu = nn.ReLU()
print(relu(torch.tensor([-2.0, 0.0, 3.5])))
```

## Building a network

A neural network is a class that inherits from `nn.Module` (Unit 14). In `__init__` you define the layers. In `forward` you describe how data flows through them. Calling the model, such as `net(x)`, runs `forward`.

network.py

```
import torch
import torch.nn as nn

class TinyNet(nn.Module):
    def __init__(self):
        super().__init__()
        self.hidden = nn.Linear(4, 8)   # 4 features in, 8 hidden units
        self.act = nn.ReLU()
        self.out = nn.Linear(8, 3)      # 3 classes out

    def forward(self, x):
        x = self.act(self.hidden(x))
        return self.out(x)

net = TinyNet()
batch = torch.rand(5, 4)                # 5 examples, 4 features each
logits = net(batch)
print(logits.shape)                     # torch.Size([5, 3])
```

The output of the last layer, before any softmax, is called the **logits**. Each row is a score for each class. The class with the highest score is the prediction.

## Parameters

parameters.py

```
import torch.nn as nn

layer = nn.Linear(4, 8)
print(layer.weight.shape)
print(layer.bias.shape)
print(sum(p.numel() for p in layer.parameters()))
```

Parameters are the weights and biases the network learns. `parameters()` lists them, and `numel()` counts the numbers in each. 4 × 8 weights plus 8 biases is 40 numbers for this layer.

**Predict the output** THINK FIRST

Shapes and gradients are where most PyTorch errors start.

```
import torch
print(torch.zeros(2, 3).shape)
```

Answer: `torch.Size([2, 3])`

```
import torch
print(torch.arange(4).sum().item())
```

Answer: `6`

```
import torch
x = torch.tensor(2.0, requires_grad=True)
y = x * x
y.backward()
print(x.grad.item())
```

Answer: `4.0`
$d(x^2)/dx = 2x = 4$ at $x = 2$.

```
import torch.nn as nn
print(nn.Linear(4, 8).weight.shape)
```

Answer: `torch.Size([8, 4])`
PyTorch stores the weight as (out_features, in_features).

# Practice

▼ **Exercise 1 — Gradient check**

Create a tensor `x` with value 5.0 and `requires_grad=True`. Compute `y = 3 * x ** 2`, call `backward()`, and print `x.grad`. Check that it matches the derivative 6x by hand.

▼ **Exercise 2 — Shape the network**

Change the `TinyNet` above so its hidden layer has 16 units. Predict the new parameter count before you run it, then check. Hint: a Linear layer from *a* to *b* has $a \times b + b$ parameters.

**Discussion**

1. Why does a network need a non-linear activation? What would happen to three stacked linear layers without one?
2. Gradients tell you which way to move a weight. What would go wrong if the gradient were always zero?
3. Why might a team prefer a smaller network that is easier to explain, even if a bigger one is slightly more accurate?

## Key terms

**Tensor**

A multi-dimensional array in PyTorch that can run on a GPU and track gradients.

**dtype**

The number type stored in a tensor, such as `float32` or `int64`.

**Device**

Where a tensor lives: `cpu` or an accelerator such as `cuda`.

**Gradient**

The rate at which an output changes as a parameter changes. Training moves parameters against the gradient.

**Autograd**

PyTorch's automatic differentiation: it records operations and computes gradients on `backward()`.

**Neuron**

A unit that computes a weighted sum of its inputs plus a bias, then applies an activation function.

**Activation function**

A non-linear function applied after a layer, such as ReLU, so the network can learn curves.

**nn.Module**

The base class for PyTorch models. Layers are defined in `__init__` and the data flow in `forward`.

**Logits**

The raw, unnormalized scores a classifier outputs before a softmax or other probability step.

**Parameters**

The learned numbers of a model, such as weights and biases.

See all terms in the glossary →